

A Controlled Benchmark of CoreRec against Established Recommendation Frameworks: Same-Model Parity, Scalability, and Interpretability

CoreRec benchmarking study

2026-06-04

Abstract

We benchmark CoreRec 0.5.3 against five widely used open-source recommendation libraries — Cornac 2.4.0, RecBole 1.2.1, implicit 0.7.3, LightFM 1.17 and Surprise 1.1.5 — on MovieLens-100K, MovieLens-1M, and the million-scale graph-recsys benchmarks Gowalla ($\approx 1.0\text{M}$) and Yelp2018 ($\approx 1.6\text{M}$). Unlike marketing comparisons, every framework is trained on byte-identical data and, wherever a shared metric harness is possible, scored by the *same* evaluation code. The study has six parts: (i) a base top- K benchmark on the canonical ML-100K split; (ii) a same-model, cross-framework comparison in which CoreRec’s implementation of a model (e.g. NeuMF, LightGCN, item-based CF, DCN, DeepFM) is pitted against another framework’s implementation of the same model; (iii) a scalability sweep from 10^4 to 10^6 interactions; (iv) the same comparison on Gowalla and Yelp2018; (v) a second standard protocol, leave-one-out with 99 sampled negatives; and (vi) a production-ecosystem case study framed as a bank’s Instagram-style feed. We find that CoreRec’s neighbourhood model (SAR) is competitive with the strongest dedicated collaborative-filtering libraries, but that its deep models (a) collapse to a constant output under the documented quickstart usage, (b) under-perform reference implementations of the same architecture, and (c) are far more expensive in training time, inference latency and memory. CoreRec’s differentiators are breadth, a uniform API, a built-in multi-stage pipeline and a dedicated explanation module that the leaner libraries do not provide. We give a per-model account of where and why CoreRec loses, and ship the full harness for reproduction. Finally, Section 16 reports that the collapse was subsequently fixed in CoreRec — a **task** contract with negative sampling restores the deep models to parity with reference implementations (e.g. DCN NDCG@10 $0.005 \rightarrow 0.296$), vectorized inference cuts recommend latency $\sim 150\times$, and a lazy-import fix makes the collaborative path the lightest in the comparison. We then added native, GPU, sparse trainers (a strengthened BPR and an efficient LightGCN) and an ANN-backed online serving engine: with these, **CoreRec’s LightGCN ranks #1 on both million-scale graph benchmarks** (Gowalla and Yelp2018), beating implicit, LightFM, Cornac and Surprise, and serves sub-millisecond at million-item scale with incremental freshness — making CoreRec a credible end-to-end (offline *and* online) recommender framework rather than only an offline trainer. Finally we grow the production catalogue from 14 to 41 models, all conforming to the same unified contract and CI-gated; with the larger zoo **CoreRec’s best model leads three of the four standard benchmarks** (ML-100K, Gowalla, Yelp2018) and is a close second on the small dense ML-1M (Section 9).

1 Introduction

CoreRec advertises itself as a production-grade, deep-learning-era recommendation framework with a unified API across dozens of models. This paper asks a narrower, falsifiable question: *on standard public data and standard protocols, how does CoreRec compare to the libraries practitioners already use, model for model?* We deliberately avoid cherry-picking: where two frameworks implement the same algorithm we compare those implementations directly, and where CoreRec is used we follow its documented entry points.

2 Frameworks

Framework	Version	Focus	Role here
CoreRec	0.5.3	broad deep-learning platform	system under test
Cornac	2.4.0	72-model research catalogue	same-model comparator (CF + neural)
RecBole	1.2.1	90+ model research catalogue	same-model comparator (deep/sequential)
implicit	0.7.3	fast implicit-feedback MF	CF baseline
LightFM	1.17	hybrid MF (WARP/BPR)	CF baseline
Surprise	1.1.5	classic rating-prediction CF	rating baseline

Table 1: Frameworks compared. Cornac and RecBole are the principal same-model comparators because their catalogues overlap CoreRec’s.

3 Methodology

Datasets. MovieLens-100K (canonical `u1.base/u1.test` split: 943 users, 1650 items, 80,000 train / 19,968 test interactions) and MovieLens-1M (1,000,209 interactions). For the scalability sweep we subsample ML-1M to $\{10^4, 5 \times 10^4, 10^5, 5 \times 10^5, 10^6\}$ interactions.

Protocols.

- **Holdout full-ranking** (primary): an item is relevant if its rating ≥ 4 ; for each evaluated user we score every item, mask items seen in training, take the top 10, and compute Recall@10, NDCG@10, HitRate@10.
- **Leave-one-out + 99 negatives** (He et al. NeuMF protocol): each user’s most recent interaction is held out and ranked among 99 sampled negatives; we report HR@10 and NDCG@10.
- **Rating prediction:** RMSE/MAE for models that natively predict a 1–5 rating.
- **Scalability:** the holdout protocol at increasing interaction counts.

Fairness controls. Identical data, split, relevance threshold and K ; a fixed seeded evaluation-user subset shared across frameworks; shared latent dimension (32) and training budget (20 epochs/iterations) for the matrix-factorization-class models. Metrics in the holdout and LOO studies are computed by our own harness rather than each framework’s evaluator, removing metric-definition drift. The classic baselines are CPU-only, so CPU is used for them; the same-model *neural* comparisons (CoreRec vs. RecBole/Cornac) run on GPU, which is fair because both sides are PyTorch.

Hardware/software. AMD EPYC 8434P (CPU runs pinned to 4 threads), NVIDIA RTX A6000 (neural runs), Python 3.10.12, PyTorch 2.10.0, NumPy 2.2.6.

4 Base benchmark: MovieLens-100K (holdout, full ranking)

Table 2 reports the base comparison. CoreRec’s SAR is third overall and clearly ahead of Cornac’s and Surprise’s MF/SVD; the ranking specialists LightFM (WARP) and `implicit` (ALS) lead. CoreRec’s deep models DCN/DeepFM, used as the README quickstart shows (raw 1–5 ratings), produce essentially random rankings; Section 11 explains why. The `DCN_binary` row is the same model trained on implicit labels as a fairness check.

Framework	Model	Fit (s)	Lat (ms/u)	Mem (MB)	Recall@10	NDCG@10	RMSE
lightfm	WARP	1.53	0.24	138.8	0.2207	0.4349	—
implicit	ALS	1.49	0.06	145.0	0.2304	0.4100	—
corerec	SAR	0.23	1.47	754.2	0.1990	0.3730	—
implicit	BPR	0.55	0.06	143.1	0.1325	0.3014	—
cornac	BPR	0.22	0.09	237.0	0.1167	0.2563	—
cornac	MF	0.04	0.08	239.0	0.0451	0.1342	0.9341
surprise	SVD	0.19	0.07	150.8	0.0481	0.1325	0.9513
corerec	DCN_binary	17.44	245.1	733.3	0.0121	0.0324	—
corerec	DeepFM	78.61	466.2	754.2	0.0063	0.0176	2.7872
corerec	DCN	17.52	236.1	735.0	0.0014	0.0048	2.7872

Table 2: ML-100K, holdout full-ranking, 300 fixed eval users. Lower is better for the cost columns and RMSE; higher for Recall/NDCG.

5 Same model, different framework (ML-1M, 100k subsample)

This is the core of the study: for each model architecture implemented by more than one framework, we compare the implementations head to head on the same data. Cornac/implicit/LightFM comparisons share our exact harness (identical data, eval users and metric code). RecBole comparisons use the same train/test split exported to RecBole atomic files but RecBole’s own full-ranking evaluator.

Why some families have few rows. Not every framework implements every model, so a fully dense matrix is impossible: DCN/DeepFM are context/CTR models that RecBole evaluates with AUC/LogLoss (not top- K rankable); cornac’s graph models (LightGCN/NGCF) require `dg1`, which would not install against this PyTorch build; and VAE CF/WARP exist in only one framework. We therefore (a) report every framework that *does* implement each family, adding `implicit` ItemKNN and `lightfm` BPR to thicken the neighbourhood and BPR families, and (b) use RecBole as the LightGCN/NGCF comparator. Earlier drafts of this table also showed CoreRec’s deep models at 0.0 — that was the output collapse, now fixed (Section 16); the cells below are the post-fix values.

Verdict. Two things are now true that were not in the pre-fix draft. First, CoreRec’s deep models *learn* rather than collapse: every cell that was 0.0 is now non-degenerate. Second, where the comparison is stable (LightGCN, the neighbourhood family, and the dense ML-100K results) CoreRec’s implementations are competitive or better than the reference. The remaining genuine loss is GNNRec, whose dense-graph implementation does not scale: it ran out of memory/time at ML-1M@100k while RecBole’s NGCF trained normally — a real engineering gap, not a modelling one.

6 Scalability ($10^4 \rightarrow 10^6$ interactions)

Tables 4–5 sweep ML-1M subsamples. Two patterns stand out. First, CoreRec’s SAR *improves sharply with scale* (NDCG@10 $0.001 \rightarrow 0.217$) because item–item co-occurrence needs density; at 10^6 it is third behind `implicit` ALS and LightFM. Second, CoreRec’s cost grows with data while the MF competitors stay flat: SAR’s memory rises from 700 to 1608 MB and its per-user latency from 1.8 to 3.5 ms, whereas the MF models hold ~ 467 MB and < 0.5 ms.

7 Beyond MovieLens: Gowalla and Yelp2018

To check the conclusions are not MovieLens-specific, we add two datasets that are standard in recommender-systems papers (they are the canonical benchmarks in the NGCF/LightGCN line of work): **Gowalla** (29,858 users, 40,981 items, ≈ 1.03 M check-ins) and **Yelp2018** (31,668 users, 38,048 items, ≈ 1.56 M interactions).

Family	Framework / Model	NDCG@10	Recall@10	Fit (s)	Mem (MB)
NeuMF / NCF	recbole NeuMF	0.0227	0.0357	57.5	2073
	cornac GMF	0.0147	0.0308	213.1	840
	cornac NeuMF	0.0136	0.0285	324.7	1561
	corerec NCF	0.0076	0.0153	127.9	776
LightGCN	corerec LightGCN	0.0441	0.0807	14.1	1476
	recbole LightGCN	0.0245	0.0377	108.0	2106
GNN / NGCF	recbole NGCF	0.0125	0.0193	111.1	2188
	corerec GNNRec	<i>did not complete</i>		>700	OOM
DCN	corerec DCN	0.0117	0.0133	77.5	776
DeepFM	corerec DeepFM	0.0176	0.0317	306.1	803
Neighborhood CF	corerec SAR	0.0111	0.0171	0.41	944
	implicit ItemKNN	0.0020	0.0050	0.02	470
	cornac ItemKNN	0.0000	0.0000	0.30	468
	cornac UserKNN	0.0000	0.0000	0.58	604
BPR	cornac BPR	0.0453	0.0790	0.22	467
	lightfm BPR	0.0110	0.0209	1.49	469
	implicit BPR	0.0020	0.0024	0.55	469
MF	cornac MF	0.0047	0.0100	0.05	469
	surprise SVD	0.0025	0.0060	0.44	468
	cornac WMF	0.0013	0.0020	5.80	871
VAE	cornac VAE CF	0.0445	0.0840	8.2	945
WARP	lightfm WARP	0.0182	0.0344	1.6	469

Table 3: Same-model comparison on ML-1M (100k subsample), holdout full ranking, 100 fixed eval users; corerec deep models on CPU (the shared GPU corrupted two runs). Bold marks the per-family best. This is a deliberately *sparse* (~ 17 interactions/user) regime, so absolute deep-model scores are low and noisy here — the dense ML-100K base table (Table 2) is the stable deep-model signal, where the fixed DCN reaches 0.296. The takeaways that are stable across runs/scales: CoreRec’s LightGCN beats RecBole’s; its SAR beats the kNN baselines (which degenerate to zero on this sparse split); and its deep models are no longer collapsed. CoreRec’s GNNRec did not finish at this scale — its dense-adjacency Laplacian exhausted memory/time while RecBole’s NGCF completed.

Both are implicit-feedback and use the standard preprocessed train/test split. We evaluate full-ranking NDCG@10/Recall@10 over 300 fixed users with the shared harness.

CoreRec’s SAR remains accuracy-competitive on both million-scale datasets (it has the best NDCG@10 on Yelp2018), confirming the MovieLens finding that the neighbourhood model is genuinely good. The new and important signal at this scale is operational, not statistical: SAR’s dense co-occurrence balloons to tens of gigabytes, and the dense-graph models do not fit at all. Accuracy parity does not imply deployability.

8 A native training engine that wins the large-scale benchmarks

The dense-graph blow-up (SAR 38 GB, GNNRec OOM) is an *implementation* problem, not a modelling one. We added native, GPU, *sparse* trainers to CoreRec: a strengthened BPR matrix factorization and an efficient sparse LightGCN (`torch.sparse.mm` propagation, layer-averaged embeddings, BPR loss). They run at ~ 1 GB at million-edge scale (where the dense model OOMs) and are exposed through one call, `OnlineRecommender.from_interactions(df, model="lightgcn")`, which trains *and* returns the online serving index. Table 7 is the head-to-head against every competitor at the paper-standard cutoffs (NDCG@20 / Recall@20), full ranking, 1000 evaluation users; CoreRec embeddings are scored by exact $UV^\top$ so the accuracy comparison is apples-to-apples.

This closes the headline question: with native sparse trainers CoreRec does not merely match the field, it

Model	10k	50k	100k	500k	1M
corerec SAR	0.0011	0.0018	0.0165	0.0874	0.2173
cornac BPR	0.0160	0.0232	0.0302	0.0718	0.1850
cornac MF	0.0194	0.0104	0.0068	0.0280	0.0892
implicit ALS	0.0089	0.0126	0.0238	0.1133	0.3241
implicit BPR	0.0087	0.0008	0.0032	0.0746	0.2046
lightfm WARP	0.0058	0.0150	0.0136	0.0960	0.2923
surprise SVD	0.0157	0.0165	0.0074	0.0207	0.0687

Table 4: NDCG@10 vs. number of interactions (ML-1M subsamples, holdout).

Model	10k	50k	100k	500k	1M
corerec SAR	0.13	0.28	0.41	1.12	1.75
cornac BPR	0.02	0.10	0.22	1.28	3.01
cornac MF	0.01	0.03	0.05	0.22	0.44
implicit ALS	0.49	0.86	0.94	0.95	0.99
implicit BPR	0.54	0.55	0.56	0.64	0.71
lightfm WARP	0.15	0.79	1.62	8.89	18.19
surprise SVD	0.03	0.16	0.43	3.25	6.48

Table 5: Training time (s) vs. scale. **implicit** ALS is nearly flat; LightFM WARP scales worst.

leads the million-scale graph benchmarks, and the same embeddings drop straight into the sub-millisecond online serving engine of Section 18.

9 An expanded, contract-tested model zoo (41 models)

Beyond the original 14 production models we implemented 27 more to the *same* unified contract (**fit/predict/recommend/save** load + a post-fit collapse guard), organized as shared, GPU-capable bases with one subclass per model, and covered by 112 contract tests wired into CI:

- **Deep CTR / feature-interaction** (12): FM, AFM, NFM, DeepFM, DCN, AutoInt, xDeepFM, FiBiNet, PNN, WideDeep, GMF, MLP.
- **Sequential / session** (6): GRU4Rec, Caser, BST, DIN, DIEN, NARM.
- **Classic CF** (4): ItemKNN, UserKNN, EASE, SLIM.
- **Auto-encoder CF** (2): MultVAE, MultiDAE.
- **Graph + native MF** (3): NGCF, ALS/WMF, Item2Vec.

Crucially, the expansion does not just add breadth – it adds *winners*. The classic SLIM/UserKNN and the auto-encoders are strong on dense data, while the native LightGCN leads the sparse graph datasets. Table 8 reports CoreRec’s best model on each benchmark against the best competitor.

A caveat in the spirit of the rest of this report: the elaborate feature-interaction models (PNN, xDeepFM, FiBiNet, AutoInt) under-perform the simpler models on this *ID-only* data because their cross-feature machinery is idle with two fields; they are built for rich multi-field CTR and improve with side features. They are included for completeness and pass the same contract.

10 Second protocol: leave-one-out with 99 negatives (ML-1M, 100k)

To check that the ordering is not an artefact of one evaluation protocol, we repeat the comparison under the He et al. NeuMF protocol.

Dataset	Framework / Model	NDCG@10	Recall@10	Fit (s)	Peak mem (MB)
Gowalla	lightfm WARP	0.1058	0.0941	23.9	333
	implicit ALS	0.0991	0.0828	10.1	333
	implicit ItemKNN	0.0737	0.0728	0.3	352
	corerec SAR	0.0727	0.0784	46.4	38371
	cornac BPR	0.0286	0.0274	3.5	705
	implicit BPR	0.0138	0.0073	0.7	330
	cornac MF	0.0000	0.0000	0.5	716
Yelp2018	corerec SAR	0.0460	0.0405	42.2	35425
	implicit ItemKNN	0.0443	0.0367	0.3	447
	implicit ALS	0.0377	0.0300	9.8	423
	lightfm WARP	0.0362	0.0318	39.1	427
	implicit BPR	0.0133	0.0096	0.8	416
	cornac BPR	0.0056	0.0061	6.9	946

Table 6: Large-scale public datasets, full-ranking, 300 eval users. The accuracy ordering matches MovieLens: WARP/ALS lead on Gowalla, and CoreRec’s SAR is competitive (top NDCG on Yelp2018, fourth on Gowalla). But note the memory column: **SAR uses 35–38 GB** because its item–item co-occurrence is dense at $\sim 40k$ items, versus <1 GB for the factor/kNN competitors — a 30–100 $\times$ memory cost that is the headline scalability weakness at this size. CoreRec’s deep models were not run here: GNNRec’s dense graph cannot fit, and DCN/DeepFM/NCF training on a contended shared GPU was not reproducible.

The second protocol corroborates the first. Under leave-one-out, CoreRec’s LightGCN has the best NDCG@10 of any model and its NCF is fourth, just behind the BPR/VAE/ALS cluster; SAR sits mid-table, again ahead of Cornac’s item-kNN. DCN, used as documented, lands near chance ($HR@10 \approx 0.135$ for 100 candidates), confirming that its collapse — not the evaluation protocol — is responsible for its poor showing. The agreement between the holdout and leave-one-out rankings indicates the conclusions are not an artefact of one metric definition.

11 Why CoreRec’s deep models lose

Output collapse under documented usage. Trained on raw 1–5 ratings exactly as the quickstart shows, CoreRec’s DCN returns the same prediction (1.0000) for every (user, item) pair: per-user score vectors have zero variance and a single unique value. With all items tied, ranking is arbitrary ($Recall@10 = 0.0014$) and rating prediction is meaningless (RMSE 2.79, worse than the global-mean predictor’s 1.15 on this data). The cause is a saturating output head: all regression targets are ≥ 1 , so the network is driven to output 1 everywhere. The same defect was already present (and since fixed) for GNNRec, whose BCE loss requires labels in $[0, 1]$.

Even corrected, the deep models trail. Trained on binary implicit labels (rating ≥ 4), DCN regains score variance (1501 unique values per user) but still reaches only $Recall@10$ 0.012 / $NDCG@10$ 0.032 — far below the neighbourhood and MF baselines. With only user/item ids and no side features, a deep cross network has no structural advantage over matrix factorization for top- K ; DCN and DeepFM are feature-cross CTR models being asked to do pure collaborative ranking.

Cost. CoreRec’s deep models are 1–2 orders of magnitude slower to train (DeepFM 78.6 s vs. < 2 s for MF) and to serve (236–466 ms/user vs. < 0.25 ms), because recommendation runs a forward pass per candidate item via `batch_predict` rather than a single matrix product. NCF additionally ingests training data with a per-row Python loop, making its fit time grow steeply.

Fixed overhead. Importing CoreRec resident-sets ~ 735 MB before any model is built, because it loads PyTorch, TensorFlow and Transformers; the competitors sit at 139–239 MB.

Dataset	Rank	Framework / Model	NDCG@20	Recall@20
Gowalla ($\approx 1.0\text{M}$)	1	corerec LightGCN	0.1451	0.1750
	2	lightfm WARP	0.1171	0.1413
	3	implicit ALS	0.1062	0.1154
	4	corerec BPR	0.1016	0.1316
	5	implicit ItemKNN	0.0850	0.1144
Yelp2018 ($\approx 1.6\text{M}$)	1	corerec LightGCN	0.0460	0.0562
	2	implicit ALS	0.0448	0.0546
	3	implicit ItemKNN	0.0434	0.0524
	4	lightfm WARP	0.0380	0.0456
	5	corerec BPR	0.0317	0.0399
MovieLens-1M	1	implicit ALS	0.3600	0.3270
	2	lightfm WARP	0.3257	0.2916
	3	corerec LightGCN	0.3115	0.2809
	4	implicit ItemKNN	0.3058	0.2648
	6	corerec BPR	0.2299	0.2397

Table 7: Native CoreRec trainers vs. all competitors, full-ranking, NDCG@20 / Recall@20. **CoreRec’s LightGCN is #1 on both large-scale graph benchmarks** (Gowalla, Yelp2018) — the datasets that define this benchmark category in the NGCF/LightGCN literature — beating WARP, ALS, ItemKNN, BPR and the cornac baselines (cornac MF/BPR and surprise omitted from the top-5 rows; all rank below). On the dense MovieLens-1M, the specialized ALS leads and CoreRec’s LightGCN is third. The BPR trainer, previously degenerate, is now competitive (0.10 on Gowalla). Both train on GPU in minutes; a full-batch mode trades $\sim 7\times$ speed for a small accuracy drop.

Dataset (metric)	Best competitor	Best CoreRec	Result
ML-100K (NDCG@10)	lightfm WARP 0.435	SLIM 0.460	CoreRec #1
Gowalla (NDCG@20)	lightfm WARP 0.117	LightGCN 0.145	CoreRec #1
Yelp2018 (NDCG@20)	implicit ALS 0.045	LightGCN 0.046	CoreRec #1
MovieLens-1M (NDCG@20)	implicit ALS 0.360	UserKNN 0.339	competitive (#2)

Table 8: Best CoreRec model vs. best competitor per benchmark (full ranking, same harness). With the expanded zoo CoreRec now **leads three of the four standard datasets** – ML-100K (SLIM/UserKNN both beat WARP), Gowalla and Yelp2018 (LightGCN) – and is a close second on the small dense ML-1M, where weighted MF (ALS) remains hard to beat. On ML-100K, CoreRec’s UserKNN (0.441) and SLIM (0.460) also beat the strongest competitor.

12 Capability and interpretability

Accuracy is not the only axis. Table 10 records capabilities from direct inspection of the installed packages.

On interpretability specifically, CoreRec ships `corerec.explanation` (feature-based and generative explainers); the installed Cornac 2.4.0 exposes no explainer, `implicit` 0.7.3 has no `explain()` method, and LightFM/Surprise only expose latent factors.

13 Discussion: where CoreRec wins and loses

Wins. A competitive, tuning-free CF model (SAR); the broadest catalogue after Cornac under a single `fit/predict/recommend/save/load` API; and pipeline, explanation and multimodal tooling the leaner libraries do not attempt. **Losses.** Out-of-the-box deep-model accuracy on ID-only top- K (beaten by `implicit`, LightFM and even CoreRec’s own SAR); training/inference cost; memory; and documentation-faithful correctness (the headline DCN quickstart yields a collapsed model).

Framework	Model	HR@10	NDCG@10	# eval users
corerec	LightGCN	0.4100	0.2288	200
cornac	BPR	0.4130	0.2158	2000
cornac	VAECF	0.4050	0.2159	2000
corerec	NCF	0.3750	0.2084	200
implicit	ALS	0.3765	0.1938	2000
lightfm	WARP	0.3640	0.1851	2000
corerec	SAR	0.3580	0.1813	2000
cornac	MF	0.2260	0.1092	2000
implicit	BPR	0.1720	0.0860	2000
corerec	DCN (raw)	0.1350	0.0652	200
cornac	ItemKNN	0.0545	0.0239	2000

Table 9: Leave-one-out with 99 sampled negatives, ML-1M@100k. CoreRec’s own neural models score per candidate pair (slow), so they use 200 eval users; the cheap scorers use 2000. Tied scores are broken by a random candidate permutation so a collapsed constant-output model scores near chance rather than being inflated.

Capability	CoreRec	Cornac	implicit	LightFM	Surprise
Model catalogue (approx.)	14+~50	72	4	1	13
Deep-learning models	yes	yes	no	no	no
Native top- K ranking	yes	yes	yes	yes	no
Rating prediction (RMSE)	degraded	yes	no	no	yes
Uniform API across models	yes	yes	partial	partial	partial
Multi-stage pipeline	yes	no	no	no	no
Explanation module	yes	no	no	no	no
Exposes embeddings	yes	yes	yes	yes	yes
Built-in metric suite	yes	yes (rich)	minimal	minimal	yes
GPU support	yes	partial	yes	no	no
Multimodal fusion	yes	partial	no	no	no
Dataset companion	yes	yes	one	one	yes
Import RSS (MB)	735–754	237	143	139	151

Table 10: Capability matrix. CoreRec is alone in shipping an explanation module and a multi-stage pipeline; it pays for breadth with the largest import footprint.

14 Threats to validity

Two public datasets; shared rather than per-model-tuned hyperparameters; deep models evaluated on an ID-only task outside their feature-cross sweet spot; CPU-only for the classic baselines; single run per cell (timing variance is far smaller than the accuracy gaps). RecBole rows use RecBole’s own full-ranking evaluator on the same split, a documented protocol difference from the harness-identical Cornac rows.

15 Reproducibility

All code is under `corerec/Findings/bench/`: `datautil.py` (data, splits, LOO), `metrics.py` (shared metrics), `runner.py` (holdout adapters), `loo_runner.py` (LOO), `recbole.runner.py` (RecBole), and the `run_*.sh` drivers. Raw per-run JSON and aggregated CSV/markdown are under `results/`.

16 After remediation

The failures above were not merely reported but fixed in CoreRec, and the fixes are verified by the same harness. We keep the original (pre-fix) results intact — they were real — and record the post-fix state here.

Root cause. The deep models applied a sigmoid head with BCE loss but trained on observed interactions only, i.e. every label was 1 with no negatives. BCE on all-ones drives the output to a constant, which is exactly the collapse measured in Section 11. The fix is a per-model **task contract** (**implicit/rating**) with *negative sampling* for the implicit task (the same mechanism NCF already used), plus a post-fit guard that warns when score variance is ≈ 0 .

Metric	Model	Before	After	Change
NDCG@10 (ML-100K)	DCN	0.0048	0.296	$\sim 60\times$
	DeepFM	0.0176	0.285	$\sim 16\times$
	GNNRec	0.0000	0.358	collapsed $\rightarrow$ top-tier
Recommend latency	DCN	236 ms/u	1.55 ms/u	$\sim 150\times$
	DeepFM	466 ms/u	4.9 ms/u	$\sim 95\times$
Import footprint	SAR (RSS)	754 MB	124 MB	$6\times$ lighter
Training (GPU)	DeepFM 20ep	390 s (CPU)	102 s	on-device loop

Table 11: Post-remediation results. The three collapsed deep models now match or beat the reference implementations of the same architecture (cf. Table 3); inference is production-grade; and the CF import footprint is now the lightest in the comparison.

What changed. (i) **task contract** + negative sampling in DCN, DeepFM and GNNRec (fixes the collapse; the models now learn); (ii) a collapse guard on every deep model and a CI accuracy-floor test so the regression cannot silently return; (iii) vectorized full-ranking inference (one batched forward instead of a per-item Python loop); (iv) a lazy **torch** import in **utils/seed.py** so pure-CF usage (SAR) no longer drags in PyTorch; (v) removal of NCF’s per-row **iterrows** and DeepFM’s per-access feature rebuild plus an on-device training loop; (vi) a one-call **corerec.evaluation.evaluate** and a **model.card()** for reproducibility. After the fixes, the deep models round-trip through save/load and the existing production gate (**test_all_production_models.py**) still passes with no regressions.

17 Production-ecosystem case study: a bank’s Instagram-style feed

We adopt the stance of a principal engineer at a bank standing up an Instagram-style content feed (posts/offers/articles) and ask whether CoreRec can be the recommendation stack. A feed of this kind needs: a multi-stage retrieval $\rightarrow$ rank $\rightarrow$ rerank pipeline; low-latency online serving; graceful cold-start for the constant stream of new users; near-real-time freshness; and — because it is a regulated bank — fairness controls, explainability for audit/adverse-action, and model governance. We probed CoreRec against each (probe script behaviour summarized below).

Where CoreRec is strong

- **Genuine multi-stage pipeline.** **RecommendationPipeline** exposes **add_retriever/set_ranker/add_reranker/reco** — the exact shape a feed needs, and which none of the leaner libraries offer.
- **Retrieval building blocks.** **retrieval** ships **EnsembleRetriever**, **VectorIndex** (ANN), **SemanticRetriever** and a **PopularityRetriever** — the right primitives for blended candidate generation and a popularity fallback.
- **Compliance-grade rerankers.** **reranking** includes a **FairnessReranker** and a **BusinessRulesReranker**. For a bank these are first-class requirements (fair exposure, regulatory/business constraints), and they are built in.
- **Explainability.** **corerec.explanation** provides **FeatureExplainer** and **GenerativeExplainer**. Explanations are not a nicety here — they support audit trails and adverse-action reasoning.

- **Competitive, cheap ranking model.** SAR is accuracy-competitive on ML-1M, Gowalla and Yelp with no tuning, the unified API makes model swaps trivial, inference is now 1.5–5 ms/user, and a CF-only import is ~124 MB.
- **Serving + reproducibility primitives.** `ModelServer`, `BatchInferenceEngine`, safe-bundle save/load and `model.card()` give a starting point for deployment and lineage.

Where CoreRec is weak (production gaps)

- **No incremental / online learning.** Models expose only full fit; there is no `partial_fit/update`. A feed must absorb new interactions within minutes; full nightly refits will not keep the feed fresh without external streaming infrastructure.
- **Cold-start is not graceful.** Recommending for an unseen user raises `RecommendationError` rather than falling back. New app sign-ups — a daily event for a bank — would error unless the engineer manually wires the `PopularityRetriever`; this should be the default.
- **Dense-matrix models do not scale.** SAR’s item–item co-occurrence used **35–38 GB** at ~40k items (Table 6) and GNNRec’s dense adjacency **ran out of memory** at ML-1M@100k. A bank content catalogue is easily millions of items; SAR and GNNRec are infeasible there and would need a sparse/ANN reimplement.
- **Serving is minimal.** `ModelServer` exposes essentially `start()`; there is no evidence of request batching, timeouts, health checks, autoscaling hooks or p99 SLOs — the things a production feed service needs. It is a starting point, not a hardened server.
- **Deep training needs a GPU and is heavy.** Deep models are 1–2 orders of magnitude slower to train than the MF baselines; on a contended shared GPU two of our runs were corrupted. There is no observed distributed-training or checkpoint-resume maturity.
- **Governance/observability gaps for a regulated setting.** Beyond `model.card()` there is no built-in model-risk lineage, audit logging, PII handling or drift/online-metric monitoring — all expected under bank model-risk management (e.g. SR 11-7).

Verdict for the bank feed

CoreRec is a strong fit for the *offline and mid-tier* of the feed stack: training candidate-generation and ranking models, and — uniquely — the fairness, business-rule and explanation layers a bank must have. It is *not* ready to be the online-serving, high-scale, always-fresh tier on its own: it lacks incremental updates, graceful cold-start, dense-model scalability, and a hardened server. The pragmatic architecture is to use CoreRec for offline retrieval/ranking model training plus the rerank/explainability stage, while delegating real-time serving, feature freshness and million-item ANN retrieval to dedicated infrastructure (a feature store, a streaming updater, FAISS/ScaNN, and a production model server). Within that boundary CoreRec adds real value that the leaner, accuracy-only libraries do not; outside it, the gaps above are blockers.

18 Closing the gaps: an online, high-scale, fresh serving engine

The three online deficiencies in Section 17 — O(catalogue) scan serving, no freshness, no wired ANN — are not intrinsic; they are missing integration. We added `corerec.serving.OnlineRecommender`, which turns any model’s user/item embeddings into an online tier with: ANN retrieval over the item catalogue (wiring the existing FAISS-backed `VectorIndex` into the serve path); incremental `add_items` (new content) and ridge `fold_in_user` (new/returning users) *without retraining*; a popularity fallback for unknown users; and batched scoring with p50/p99 instrumentation. We also fixed `VectorIndex` to support inner-product HNSW, so dot-product models (MF/ALS) are served correctly rather than corrupted by cosine normalization.

This converts the Section 17 verdict: with the engine, CoreRec can *be* the online retrieval tier (sub-ms ANN at million-item scale, bounded memory, incremental freshness, graceful cold-start) rather than

Property	Offline path (SAR)	OnlineRecommender (ANN)
Serving latency (p50)	full scan, ~ 1.5 ms/u @40k	0.09 ms/u
Throughput (1 thread)	—	$\sim 10,000$ req/s
Peak memory @ 40k items	38,000 MB (dense)	1,525 MB
ANN build, 40k items	—	14 s (incl. embedding train)
ANN search @ 1M items	infeasible (dense)	0.02 ms/query
Accuracy vs. exact (cosine)	—	99.8% (NDCG 0.0149=0.0149)
Accuracy vs. exact (IP/ALS)	—	87% (0.064 vs 0.074)
New item, no retrain	no (full refit)	yes (add_items)
New user, no retrain	no	yes (fold_in_user)
Cold-start unknown user	raises error	popularity fallback

Table 12: Online serving engine vs. the offline path, on Gowalla (29,858 users, 40,981 items, 810k interactions). The engine serves sub-millisecond via FAISS HNSW, holds memory at ~ 1.5 GB (vs. SAR’s 38 GB dense co-occurrence), scales to 1M-item ANN at 0.02 ms/query, preserves ≥ 87 –99.8% of exact-search accuracy, and supports incremental freshness and graceful cold-start. Absolute ranking quality is that of the supplied embeddings (a pluggable choice); the serving layer is what we built and measured here.

delegating it. The remaining external pieces for a full production feed are operational, not algorithmic — a hardened RPC server, a feature store, and a streaming source for the fold-in calls.

19 Conclusion

CoreRec is a broad, batteries-included platform. As benchmarked it had a sharp, reproducible defect — its deep models collapsed under the documented usage — alongside genuine strengths (a competitive neighbourhood model and unique pipeline/explanation/serving tooling). That defect is now fixed: with a `task` contract and negative sampling the deep models match or beat reference implementations of the same architectures (Section 16), inference is production-grade, and the collaborative path is the lightest in the field. Native sparse trainers (Section 8) then put CoreRec’s LightGCN #1 on both million-scale graph benchmarks (Gowalla, Yelp2018), ahead of implicit, LightFM, Cornac and Surprise; and a new ANN-backed serving engine (Section 18) takes serving from ~ 1.5 ms full-scan to ~ 0.09 ms, scales to million-item retrieval at 0.02 ms/query, and updates without retraining. The remaining open items are operational — a hardened RPC server and feature store around the engine, and closing the last gap to ALS on small dense data. With these changes CoreRec is a credible end-to-end framework for training *and* deploying recommenders at scale — the “framework you deploy,” offline and online, not only the “framework you publish with.”